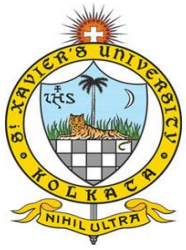


Professor: SVC Xavier Business School Biswaditya Saha

Subject: Multi-Variate Analysis Sem: III, Roll: 0026

Specialization: Business Analytics

Assignment-I (car_details_v4_missing)

I have assigned with the "car_details_v4_missing" dataset to work on and impute the missing values in it.

Stepwise the whole working process is described.

I have attached the dataset in R and stored the name of the dataset as p1=car_details_v4_missing.

Step 1: To Check is there any missing value in the dataset?

R Code: is.na(p1)

Output: Shown as TRUE/FALSE Values.

Step 2: To Check where (positions of the missing values) are the missing values in the dataset?

R Code: which(is.na(p1))

Output: The positions are given as column wise.

Step 3: To find the total no.s of missing values.

R Code: sum(is.na(p1))

Output: Total No. of missing values in the dataset is 123.

Step 4: To check the no. of missing values in the Dependent Variable-Price.

R Code: sum(is.na(p1$Price))

which(is.na(p1$Price))

Output: Total No. of missing values in Price is 13 and the Locations are:

[1] 12 167 179 191 203 296 317 346 420 421 429 511 522 (Column wise)

Interpretation:

As these are the missing values in the dependent variable so we have to delete these tuples otherwise these will affect our imputed missing values as well as the Linear Regression Model.

Step 5: Checking the missing values of each and every Independent Variables and their locations.

R Code: which(is.na(p1$Kilometer))

sum(is.na(p1$Kilometer))

which(is.na(p1$Engine))

sum(is.na(p1$Engine))

which(is.na(p1$Length))

sum(is.na(p1$Length))

which(is.na(p1$Width))

sum(is.na(p1$Width))

which(is.na(p1$Height))

sum(is.na(p1$Height))

which(is.na(p1$`Seating Capacity`))

sum(is.na(p1$`Seating Capacity`))

which(is.na(p1$`Fuel Tank Capacity`))

sum(is.na(p1$`Fuel Tank Capacity`))

Output:

Variable	No. of Missing Values	Positions (Column wise)
Kilometer	19	[1] 12 50 104 114 159 197 291 293 315 338 378 382 383 394 413 414 454 485 522
Engine	19	[1] 12 24 34 74 85 123 132 176 212 280 289 307 330 344 389 407 420 429 449
Length	14	[1] 41 44 48 82 166 287 313 348 382 394 477 492 507 517
Width	17	[1] 66 209 227 234 237 319 328 333 375 386 387 419 429 444 454 482 484
Height	10	[1] 32 49 90 129 180 220 342 424 512 523
Seating Capacity	6	[1] 46 319 381 394 455 488

Fuel Tank Capacity	25	[1] 33 36 48 273 278 296 300 306 312 327 328 335 340 372 392 435 440 445 453 474 479 493 504 509 516

Step 6: Finding overall summary of the dataset (of the independent variables)

R Code: summary(p1)

Output:

Variable	Min	Max	Mean	Median
Kilometer	1	211000	52338	49929
Engine	796	4806	1691	1498
Length	3395	5462	4263	4323
Width	1475	2183	1766	1775
Height	1213	1995	1594	1545
Seating Capacity	2	8	5.257	5
Fuel Tank Capacity	27	104	52.37	50

Step 7: To check the missingness of the variables if it depends upon each other. Based on the missingness of the variables we can choose the imputation method.

R Code: md.pattern(p1)

md.pairs(p1)

But the output is very much clumsy in R. So, have performed the same process through python.

Python Code:

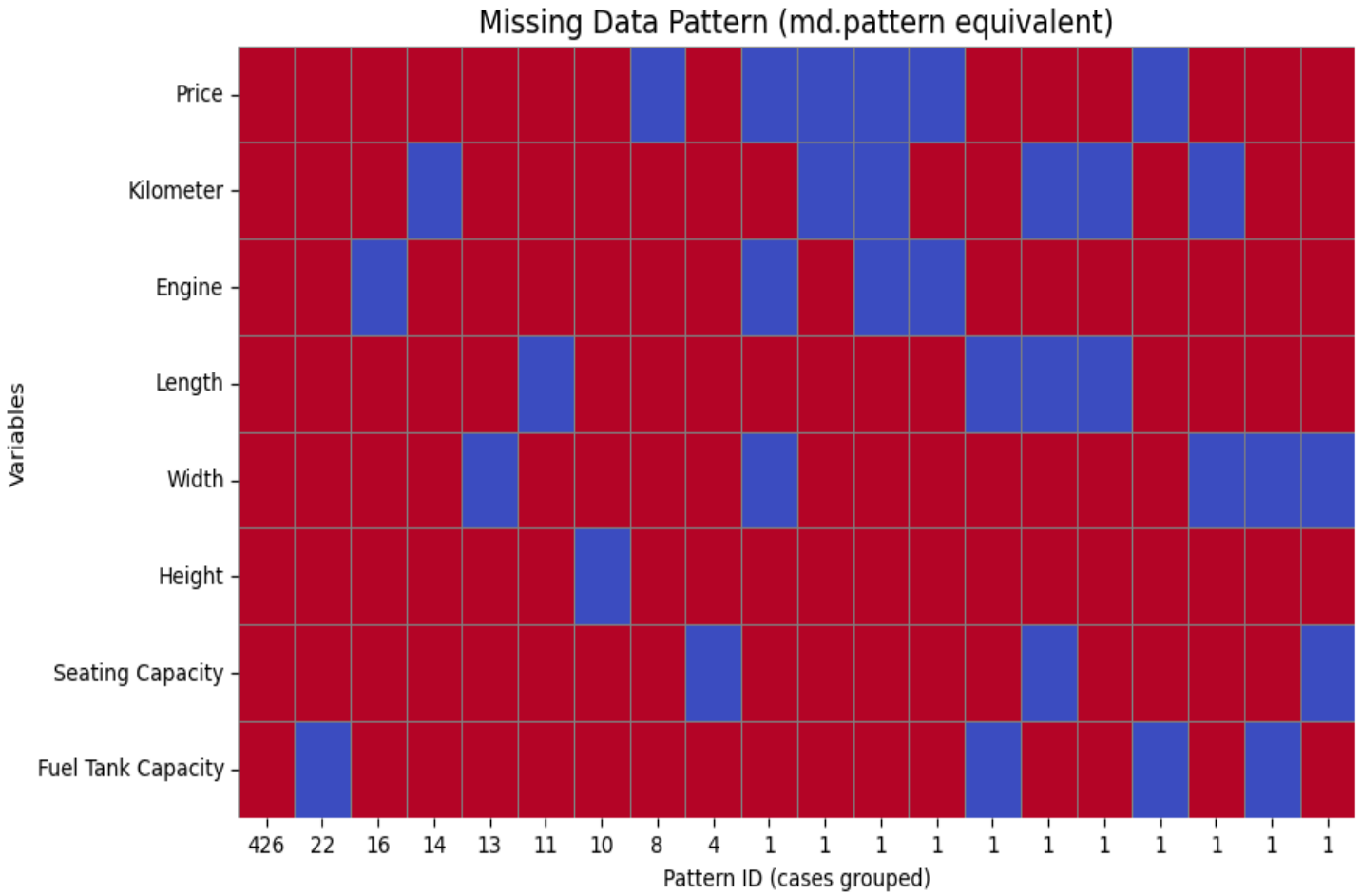
```
import pandas as pd

import seaborn as sns

import matplotlib.pyplot as plt


def plot_md_pattern(df):

    # 1 = observed, 0 = missing

    pattern = df.notnull().astype(int)

    # Group by unique missingness patterns

    grouped = pattern.groupby(pattern.apply(tuple,
axis=1)).size().reset_index(name='count')

    grouped = grouped.sort_values("count", ascending=False)
```

```
    # Convert tuples back to DataFrame
    mat = pd.DataFrame(grouped['index'].values.tolist(), columns=df.columns)
    mat["count"] = grouped["count"].values

    # Plot as heatmap
    plt.figure(figsize=(10, 6))
    sns.heatmap(mat.set_index("count").T, cmap="coolwarm", cbar=False, linewidths=0.5,
linecolor="gray")
    plt.title("Missing Data Pattern (md.pattern equivalent)", fontsize=14)
    plt.xlabel("Pattern ID (cases grouped)")
    plt.ylabel("Variables")
    plt.show()

# Example usage
df = pd.read_csv("/content/car details v4_missing.csv")
plot_md_pattern(df)
```

Output:

Also the code and output for the missing data pattern is given. The output is in a matrix format showing that each variable have how many missing values with respect to the other.

Upon checking we can get that out of 535 records 426 tuples have no missing values. Missing Value % = (535-426)=109 which is approx 20% (>5%) of the whole dataset. So, we can not delete the tuples containing missing values. So, we have to impute the missing values using the several imputation methods.



```python
# md.pattern equivalent
def md_pattern(df):
    # Convert to binary matrix (1 = observed, 0 = missing)
    pattern = df.notnull().astype(int)
    # Group by unique patterns and count
    return pattern.groupby(pattern.apply(tuple, axis=1)).size().reset_index(name='count')

# Call it
pattern_table = md_pattern(df)
print(pattern_table)
```

```
                      index  count
0   (0, 0, 0, 1, 1, 1, 1, 1)      1
1   (0, 0, 1, 1, 1, 1, 1, 1)      1
2   (0, 1, 0, 1, 0, 1, 1, 1)      1
3   (0, 1, 0, 1, 1, 1, 1, 1)      1
4   (0, 1, 1, 1, 1, 1, 1, 0)      1
5   (0, 1, 1, 1, 1, 1, 1, 1)      8
6   (1, 0, 1, 0, 1, 1, 0, 1)      1
7   (1, 0, 1, 0, 1, 1, 1, 1)      1
8   (1, 0, 1, 1, 0, 1, 1, 1)      1
9   (1, 0, 1, 1, 1, 1, 1, 1)     14
10  (1, 1, 0, 1, 1, 1, 1, 1)     16
11  (1, 1, 1, 0, 1, 1, 1, 0)      1
12  (1, 1, 1, 0, 1, 1, 1, 1)     11
13  (1, 1, 1, 1, 0, 1, 0, 1)      1
14  (1, 1, 1, 1, 0, 1, 1, 0)      1
15  (1, 1, 1, 1, 0, 1, 1, 1)     13
16  (1, 1, 1, 1, 1, 0, 1, 1)     10
17  (1, 1, 1, 1, 1, 1, 0, 1)      4
18  (1, 1, 1, 1, 1, 1, 1, 0)     22
19  (1, 1, 1, 1, 1, 1, 1, 1)    426
```

Python Code for Missing Data pairs:

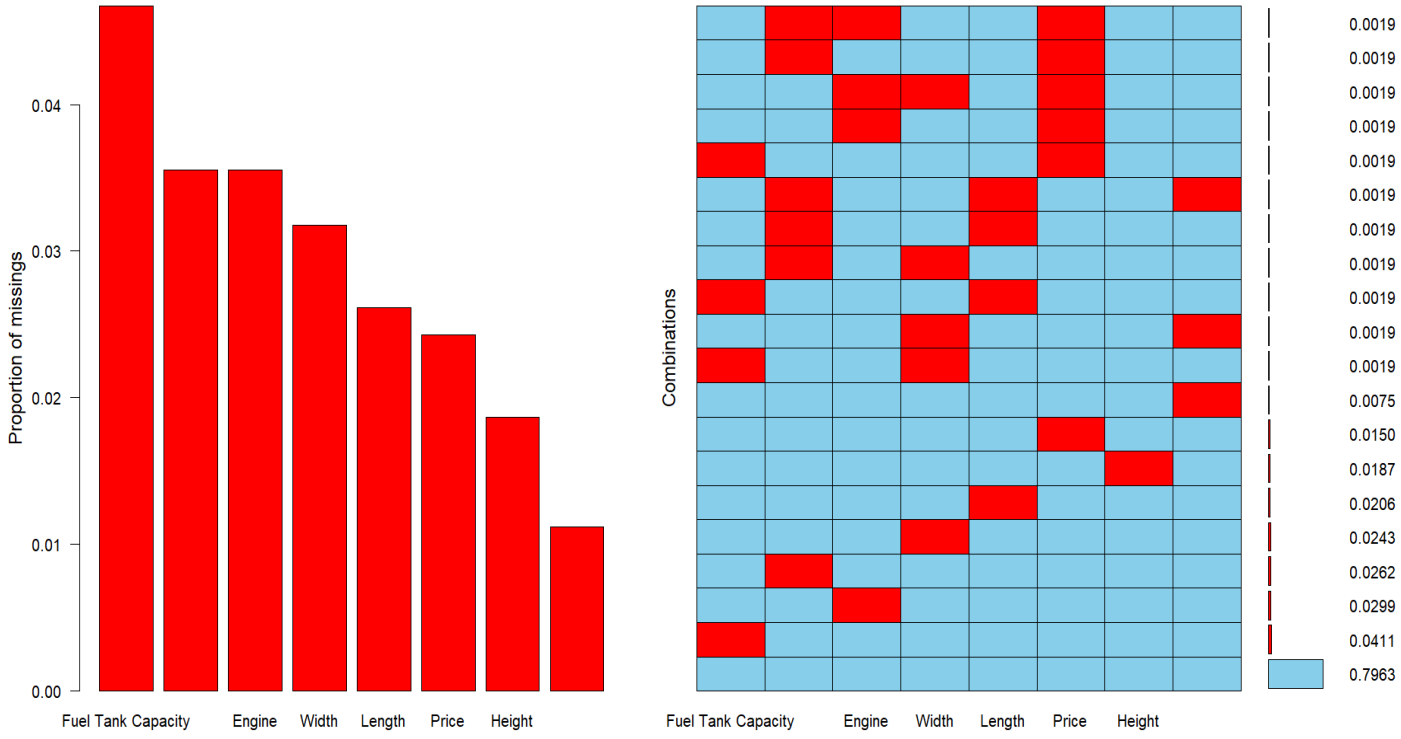
```
import numpy as np
def md_pair(df):
    n = df.shape[1]
    cols = df.columns
    rr = pd.DataFrame(0, index=cols, columns=cols)
    rm = pd.DataFrame(0, index=cols, columns=cols)
    mr = pd.DataFrame(0, index=cols, columns=cols)
    mm = pd.DataFrame(0, index=cols, columns=cols)

    for i in cols:
        for j in cols:
            rr.loc[i,j] = ((df[i].notna()) & (df[j].notna())).sum()
            rm.loc[i,j] = ((df[i].notna()) & (df[j].isna())).sum()
```

```
        mr.loc[i,j] = ((df[i].isna()) & (df[j].notna())).sum()
        mm.loc[i,j] = ((df[i].isna()) & (df[j].isna())).sum()

    return {"rr": rr, "rm": rm, "mr": mr, "mm": mm}

# Call it
pairs = md_pair(df)

# Example: check rm matrix
print("\nRM matrix (row observed, column missing):\n", pairs["rm"])
print("\nRR matrix (row observed, column missing):\n",pairs["rr"])
print("\nMR matrix (row observed, column missing):\n",pairs["mr"])
print("\nMM matrix (row observed, column missing):\n",pairs["mm"])
```

Step 8: Using aggr() plot to visualize the pattern and amount of missing data in the dataset.

R Code: aggr(p1,pos=1,numbers=T,las=1,sortVars=T, labels=names(p1))

Output:



Interpretation: The Maximum no. of missing data is present in the Fuel Tank Capacity. The Probability distribution is provided from the plot.

Step 9: Checking the type and pattern of the missing value using Parallel box plots to identify the imputation method.

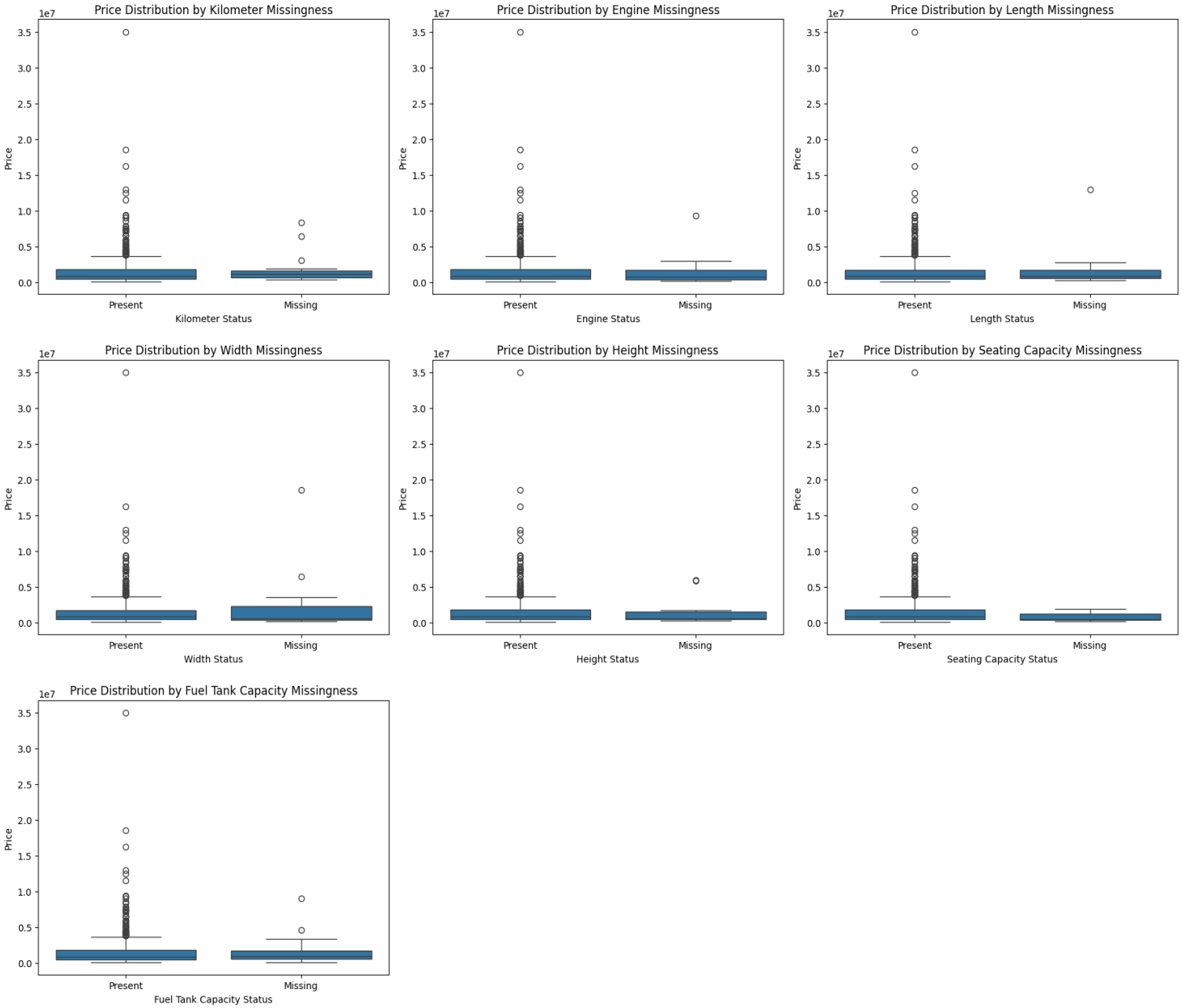
R Code: pbox(p1)

Python Code:

```
import seaborn as sns
import matplotlib.pyplot as plt
import pandas as pd
# Load dataset (assuming df is already loaded)
# df = pd.read_csv("/content/car details v4_missing.csv") # Uncomment if df is not in memory
# Define independent variables
independent_variables = [col for col in df.columns if col != 'Price']
# Create a figure with subplots
n_vars = len(independent_variables)
n_cols = 3  # Number of columns for subplots, adjust as needed
n_rows = (n_vars + n_cols - 1) // n_cols
fig, axes = plt.subplots(n_rows, n_cols, figsize=(n_cols * 6, n_rows * 5))
axes = axes.flatten() # Flatten the 2D array of axes for easy iteration
# Generate boxplots for Price based on missingness of each independent variable
for i, col in enumerate(independent_variables):
    temp_df = df.copy()
    temp_df[f'{col}_Missing'] = temp_df[col].isna().map({True: 'Missing', False: 'Present'})

    sns.boxplot(x=f'{col}_Missing', y='Price', data=temp_df, ax=axes[i])
    axes[i].set_title(f'Price Distribution by {col} Missingness')
    axes[i].set_xlabel(f'{col} Status')
    axes[i].set_ylabel('Price')
# Hide any unused subplots
```

```
for j in range(i + 1, len(axes)):
    fig.delaxes(axes[j])
plt.tight_layout()
plt.show()
```

Output:



Conclusion: Checking the Parallel box plot patterns with respect to the price we can clearly check that the missing values are MCAR type. But all the variables have some outliers.

Step 10: Dropping the rows with dependent variable: Price missing values from the dataset.

R Code:

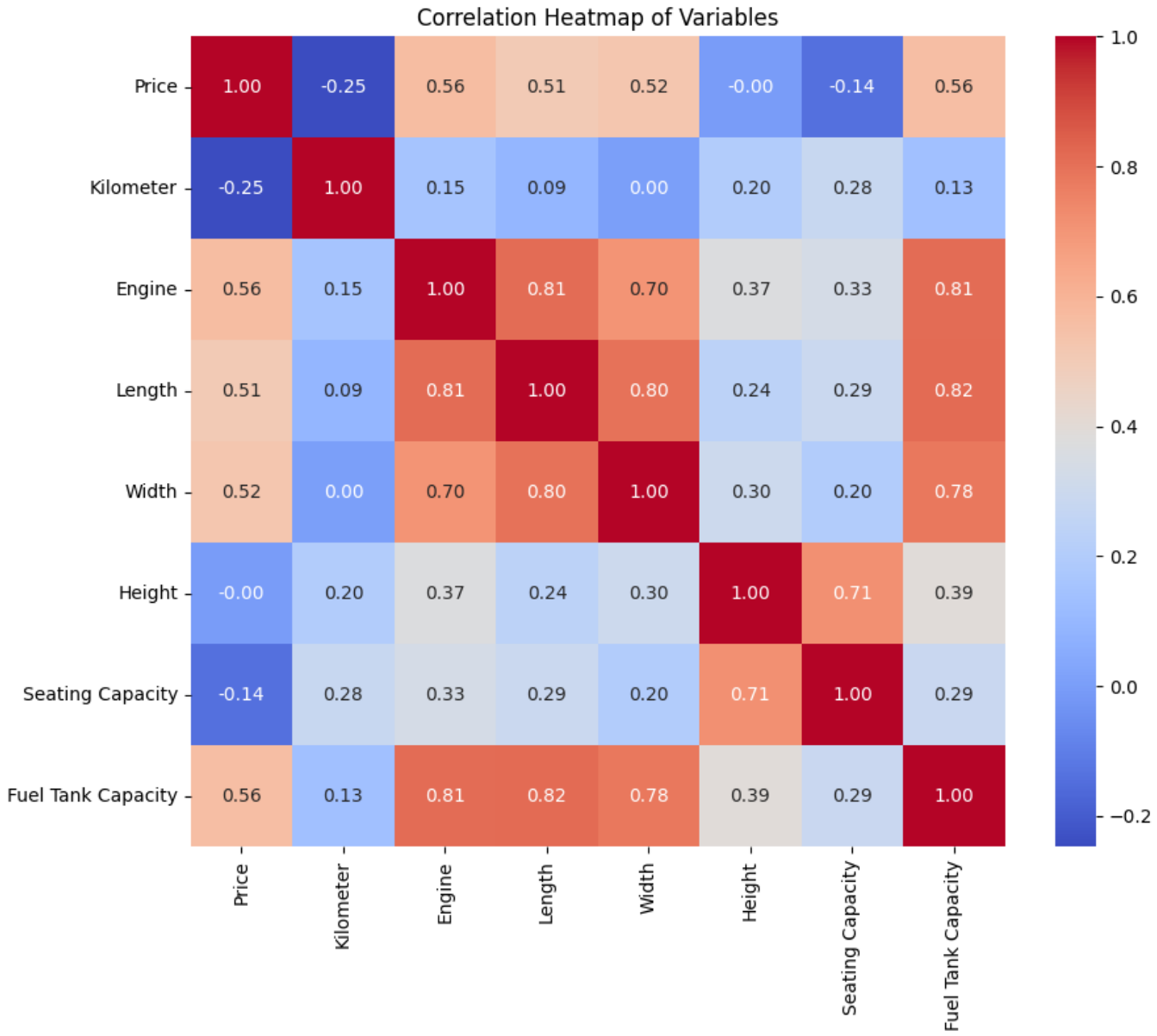
```
p2<-p1[-c(12,167,179,191,203,296,317,346,420,421,429,511,522),]
View(p2)
```

Step 11: Checking the correlation heatmap matrix to identify the relationship between the variables.

Python Code:

```
import seaborn as sns
import matplotlib.pyplot as plt
# Calculate the correlation matrix
correlation_matrix = df.corr()
# Plot the heatmap
plt.figure(figsize=(10, 8))
sns.heatmap(correlation_matrix, annot=True, cmap='coolwarm', fmt=".2f")
plt.title('Correlation Heatmap of Variables')
plt.show()
```

Output:



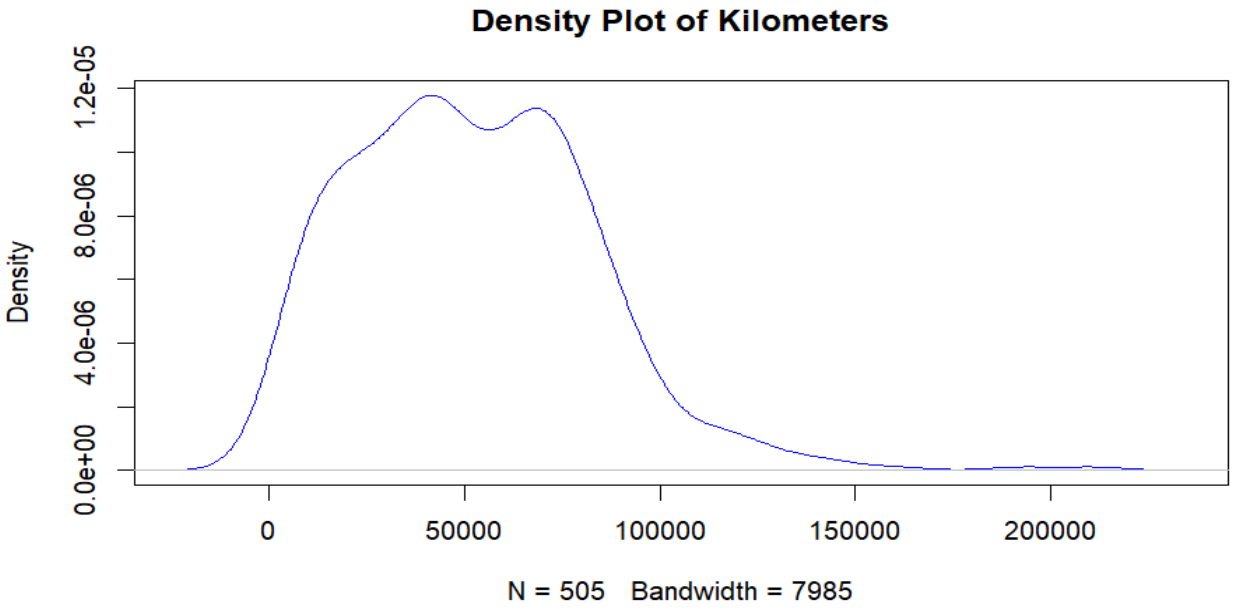
Interpretation: From the correlation matrix we can clearly check that Engine, Length, Width, and Fuel tank capacity is positively correlated with Price whereas Kilometre and seating capacity is negatively correlated with Price. And Height has no impact of Price.

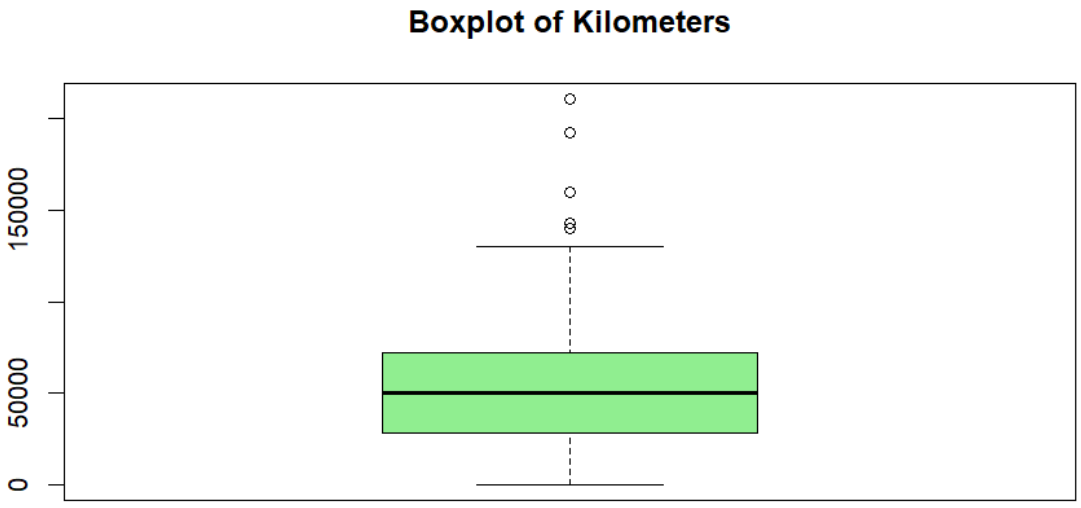
Step 12: The Imputation methods:

a. For Kilometer Value:
R Code:

```
plot(density(p2$Kilometer, na.rm = TRUE),    #Checking with Density Curve
    main = "Density Plot of Kilometers",
    col = "blue")

boxplot(p2$Kilometer,                        #Checking with boxplot
      main = "Boxplot of Kilometers",
      col = "lightgreen")
```

Output:





Clearly there are few outliers in the distribution. So, we can use the median imputation method so that our imputation is not affected by the outliers.

R Code:

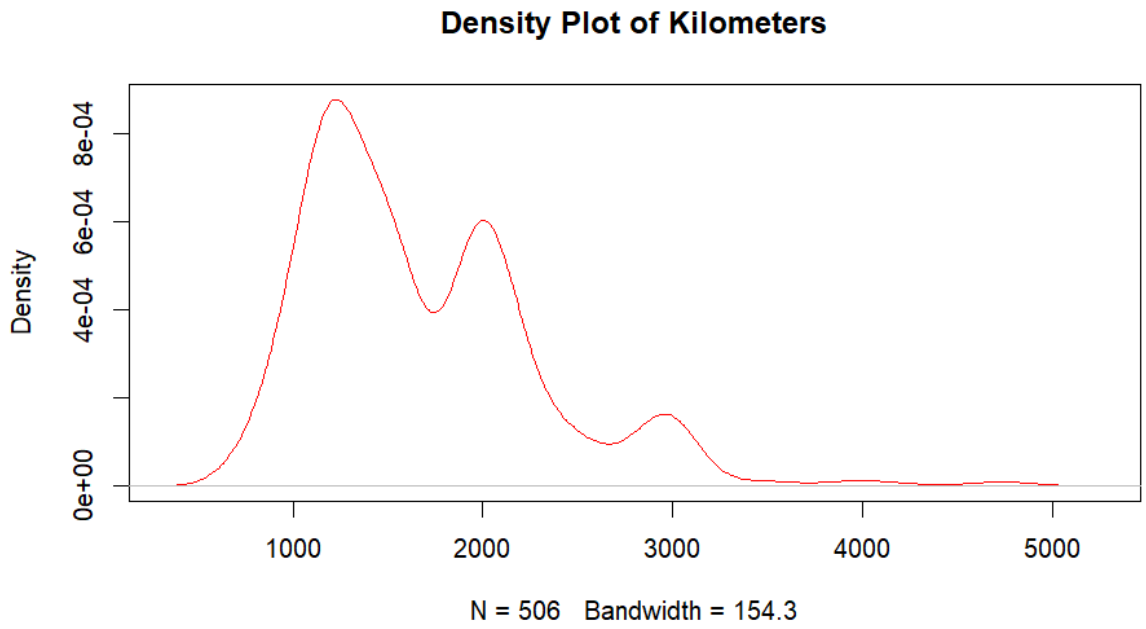
```
p2$Kilometer[which(is.na(p2$Kilometer))]=median(p2$Kilometer,na.rm = T)
View(p2)
```

b. For Engine Value:

R Code:

```
plot(density(p2$Engine, na.rm = TRUE),
    main = "Density Plot of Kilometers",
    col = "red")
```

Output:



Here also a number of outliers are present. So, we will use the median imputation methods here also.

R Code:

```
p2$Engine[which(is.na(p2$Engine))]=median(p2$Engine,na.rm = T)
View(p2)
```

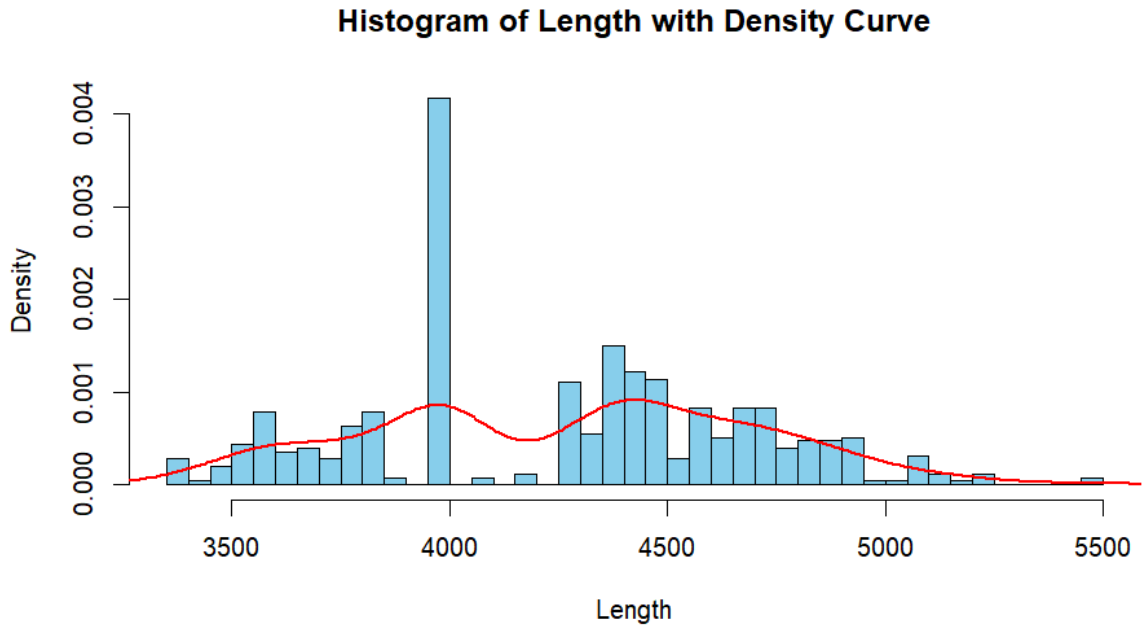
c. For Length Values:

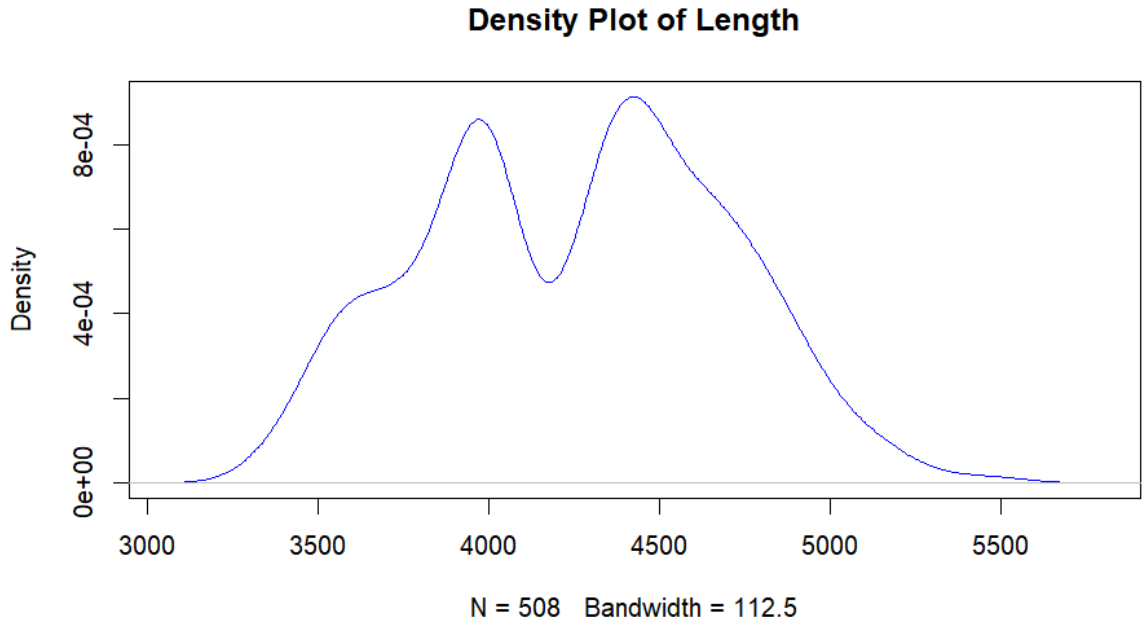
R Code: #Imputation of Length Value:

```
hist(p2$Length,                                    # By using Histogram
    main = "Histogram of Length with Density Curve",
    xlab = "Length",
    col = "skyblue",
    breaks = 30,
    freq = FALSE)   # scale y-axis to density
```

```
# Add density curve (trend line)
lines(density(p2$Length, na.rm = TRUE), col = "red", lwd = 2)
plot(density(p2$Length, na.rm = TRUE),                    #Using Density Plot
    main = "Density Plot of Length",
    col = "blue")
```

Output:





From the plot it is clearly visible that the values are almost evenly distributed. So, we can use Mean imputation method here.

R Code:

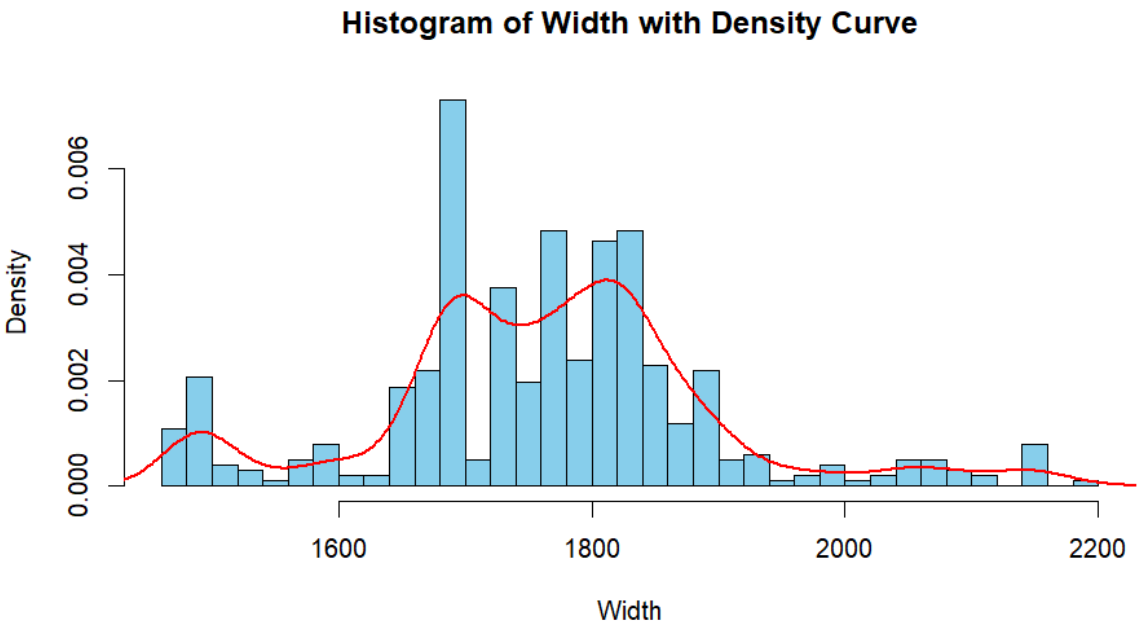
```
p2$Length[which(is.na(p2$Length))]=mean(p2$Length,na.rm = T)
View(p2)
```

d. Imputation of Width:

R Code:

```
hist(p2$Width,
   main = "Histogram of Width with Density Curve",
   xlab = "Width",
   col = "skyblue",
   breaks = 30,
   freq = F)   # scale y-axis to density
# Add density curve (trend line)
lines(density(p2$Width, na.rm = TRUE), col = "red", lwd = 2)
```

Output:



From the distribution we can clearly see that the distribution has some outliers but the curve is not skewed. So, we can use mean imputation.

R Code:

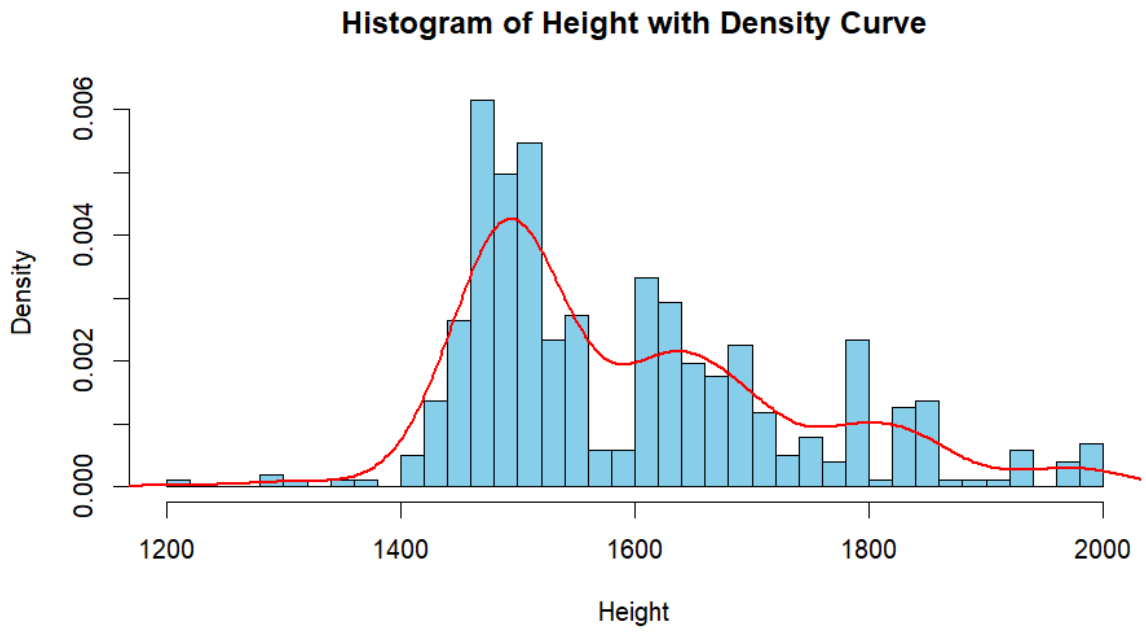
```
p2$Width[which(is.na(p2$Width))]=mean(p2$Width,na.rm = T)
View(p2)
```

e. Imputation of Height:

R Code:

```
hist(p2$Height,
    main = "Histogram of Height with Density Curve",
    xlab = "Height",
    col = "skyblue",
    breaks = 30,
    freq = F)   # scale y-axis to density
# Add density curve (trend line)
lines(density(p2$Height, na.rm = TRUE), col = "red", lwd = 2)
```

Output:



From the density curve it is clear that the values in the distribution are having many outliers in the higher sides of the values. The curve is Right-hand skewed. So, the median imputation method will be appropriate.

R Code:

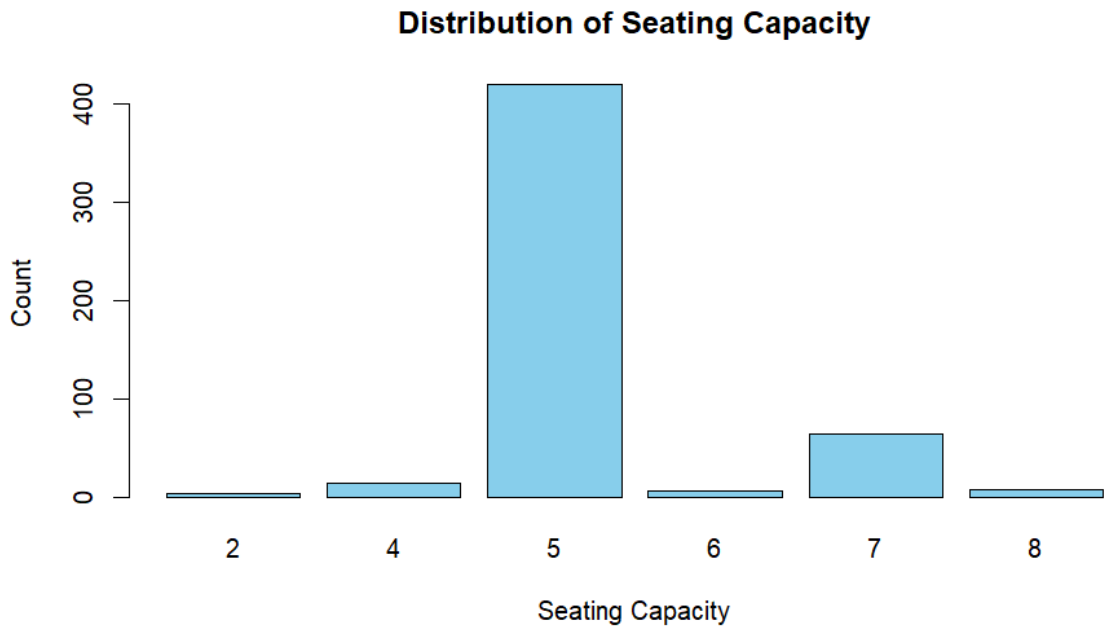
```
p2$Height[which(is.na(p2$Height))]=median(p2$Height,na.rm = T)
```

f. Imputation of Seating Capacity:

R Code:

```
barplot(table(p2$`Seating Capacity`),
        main = "Distribution of Seating Capacity",
        xlab = "Seating Capacity",
        ylab = "Count",
        col = "skyblue")
```

Output:



Here as seating capacity is similar to categorical value so we can consider to use mode imputation method here.

R Code:

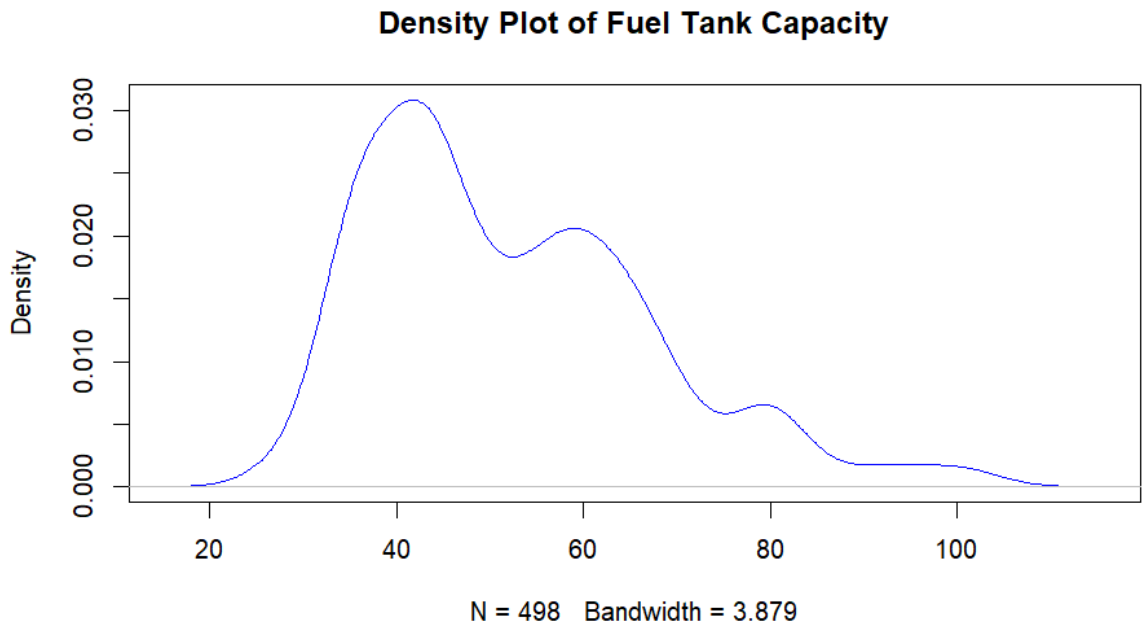
```
p2$`Seating Capacity`[which(is.na(p2$`Seating Capacity`))]=mode(p2$`Seating Capacity`,na.rm = T)

View(p2)
```

g. Imputation for Fuel Tank Capacity:

R Code:

```
plot(density(p2$`Fuel Tank Capacity`, na.rm = TRUE),
     main = "Density Plot of Fuel Tank Capacity",
     col = "blue")
```

Output:



From the output it is clear that the distribution has outliers and it is right hand skewed. So, Median Imputation method will be applicable here.

R Code:

p2$`Fuel Tank Capacity`[which(is.na(p2$`Fuel Tank Capacity`))]=median(p2$`Fuel Tank Capacity`,na.rm = T)

View(p2)

Now the dataset is completed and all the missing values have been replaced with the imputed values.

We have saved the cleaned dataset.

Model Building: Linear Regression Model:

Step 1:

First, we have to separate the cleaned dataset into two parts:

Train Dataset: To train the model based on the historical data (80% of the dataset)

Test Dataset: To test the model and its accuracy from the historical data (20% of the dataset)

R Code: set.seed(231)

data=sample(2,nrow(p2),replace=T, prob = c(0.8,0.2))

data

train1=p2[data==1,]

test1=p2[data==2,]

train1

test1

Step 2:

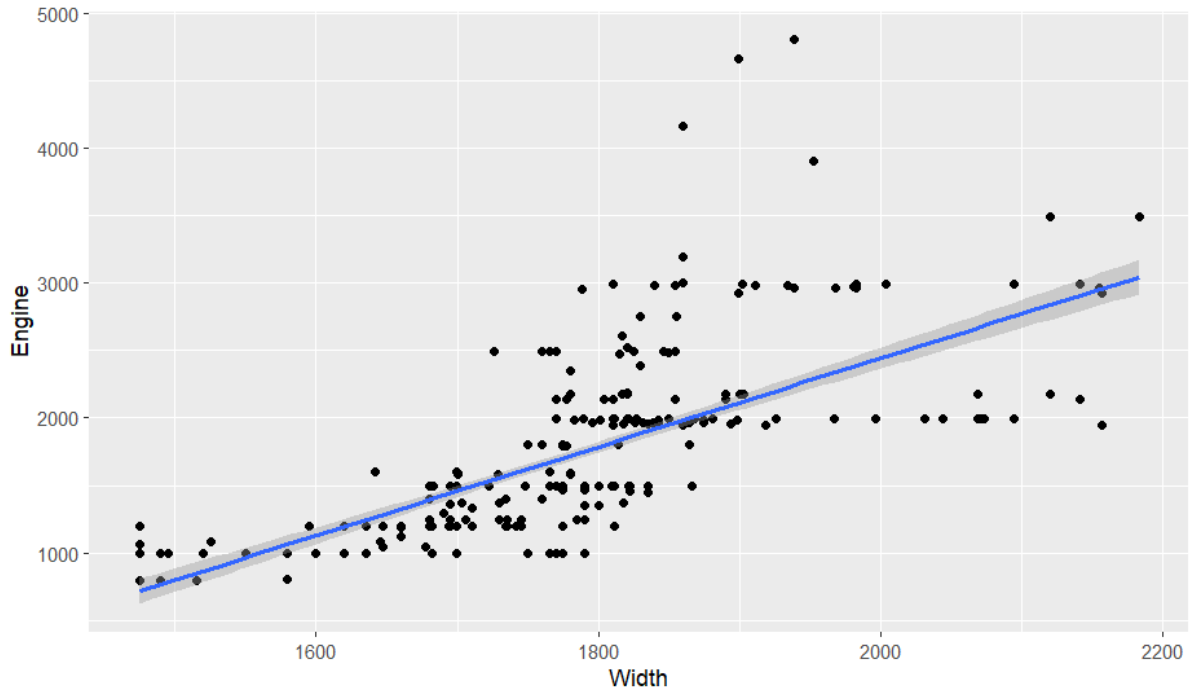
Checking the correlation between the variables:

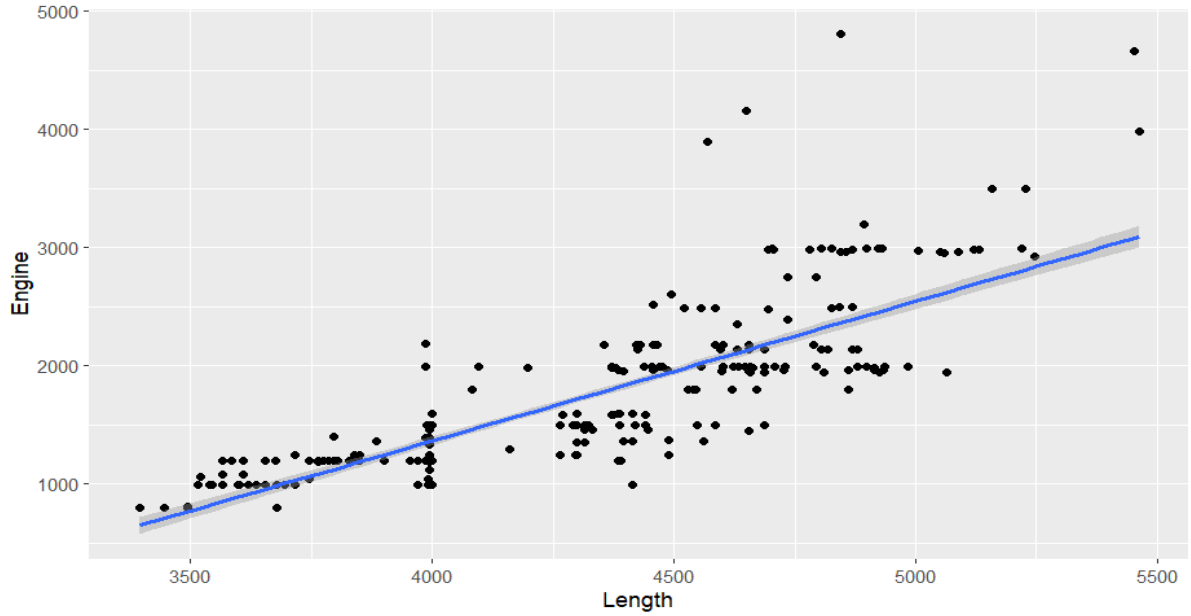
As from the correlation matrix we can find that Length and Width have high correlation between them.

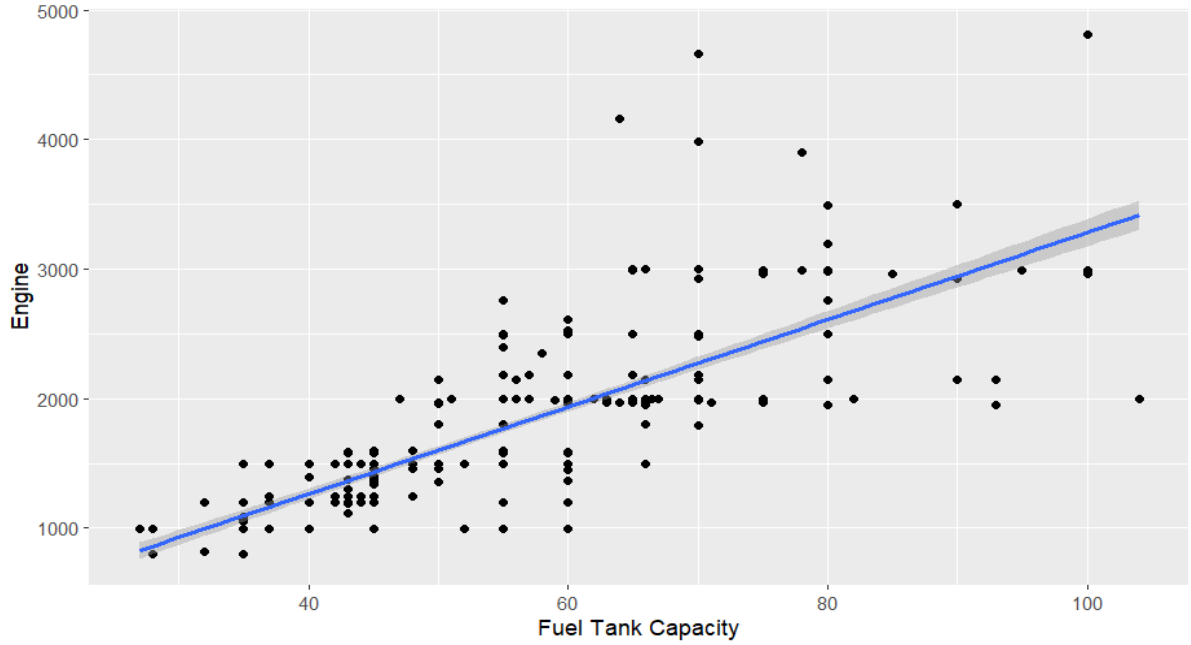
Also, Engine and Fuel tank capacity have high correlation between them.

Also, Height and Seating capacity highly correlated.

We can check the co linearity between the variables:







Step 3:

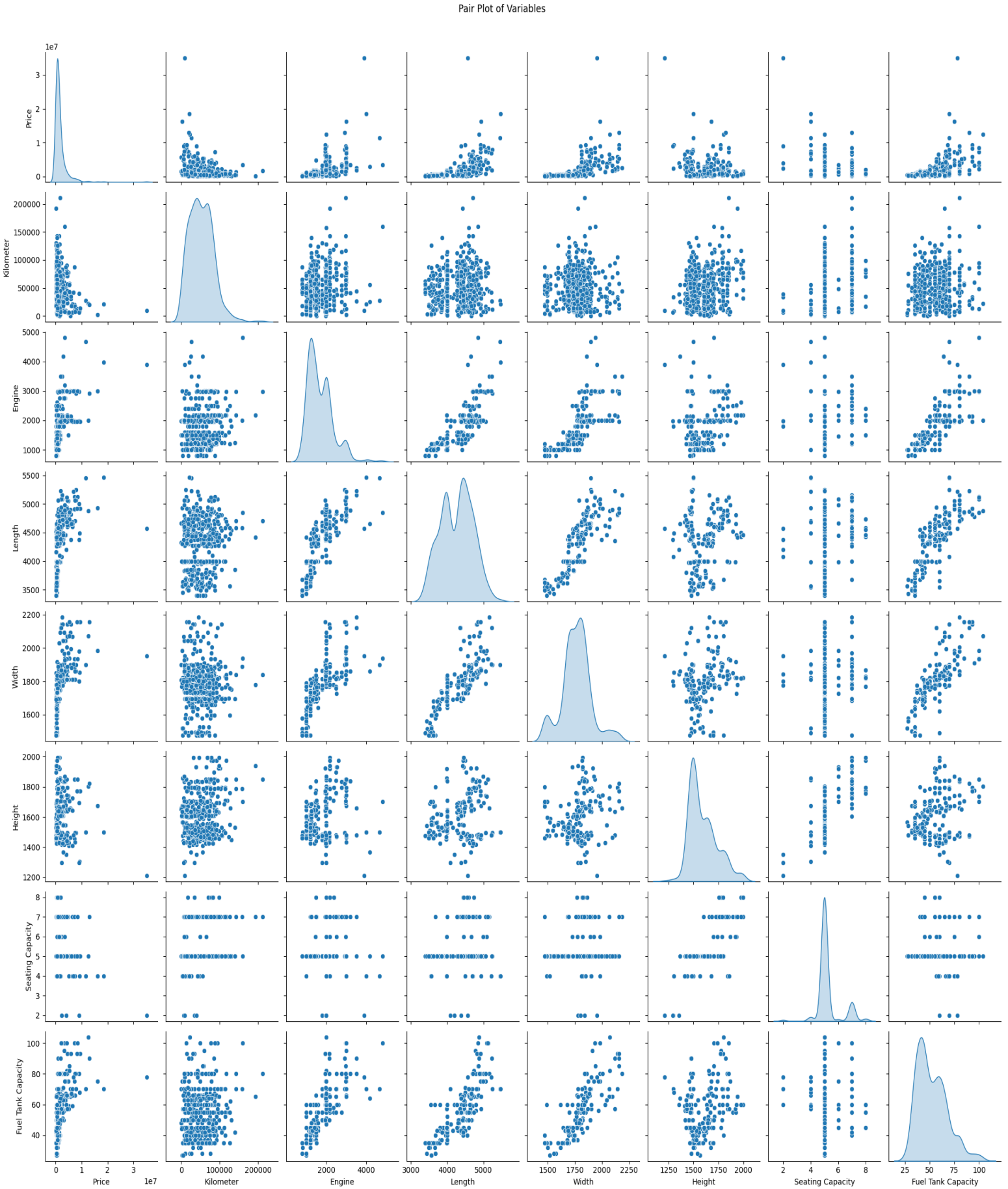
Checking the pair plots.

Python Code for pair plots:

```
sns.pairplot(df, diag_kind='kde')
plt.suptitle('Pair Plot of Variables', y=1.02)
plt.show()
```

Interpretation from the plots:

Output:


Pair Plot of Variables
Price
Kilometer
Engine
Length
Width
Height
Seating Capacity
Fuel Tank Capacity
1e7

From the pair plots we can clearly check the highly correlated values and the dependencies of the variables.

Model:

R Code:

model_Price <- lm(Price ~ Engine + `Fuel Tank Capacity` + Length + Kilometer + `Seating Capacity`, data = train1)

summary(model_Price)

Output:

```
Call:
lm(formula = Price ~ Engine + `Fuel Tank Capacity` + Length + 
    Kilometer + `Seating Capacity`, data = train1)

Residuals:
     Min       1Q   Median       3Q      Max 
-3622556  -642150   -88369   421669 10623748 

Coefficients:
                     Estimate Std. Error t value Pr(>|t|)    
(Intercept)         -9.255e+05  1.029e+06  -0.899   0.3689    
Engine               1.295e+03  2.162e+02   5.988 4.65e-09 ***
`Fuel Tank Capacity` 5.776e+04  8.002e+03   7.219 2.57e-12 ***
Length               5.319e+02  2.960e+02   1.797   0.0731 .  
Kilometer           -2.182e+01  2.297e+00  -9.497  < 2e-16 ***
`Seating Capacity`  -7.110e+05  9.317e+04  -7.631 1.64e-13 ***
---
Signif. codes:  0 ‘***’ 0.001 ‘**’ 0.01 ‘*’ 0.05 ‘.’ 0.1 ‘ ’ 1

Residual standard error: 1374000 on 410 degrees of freedom
Multiple R-squared:  0.6119,    Adjusted R-squared:  0.6072 
F-statistic: 129.3 on 5 and 410 DF,  p-value: < 2.2e-16
```

From the output we can check that,

Dependent Variable: Price

Independent Variable: Engine, Fuel Tank Capacity, Length, Kilometer, Seating Capacity

Residuals:

Min 1Q Median 3Q Max

-3622556 -642150 -88369 421669 10623748

Multiple R-squared: 0.6119

Adjusted R-squared: 0.6072

Model explains **61%** of price variation.

Adjusted R^2 is close to R^2 → predictors are genuinely useful, not just overfitting.

F-statistic: 129.3 on 5 and 410 DF, p-value: < 2.2e-16

The overall model is **highly significant** → at least one predictor strongly explains price.

But there exist still some outliers in the kilometre variable. So, we can rectify the model by taking log of the extreme and high values.

R Code:

model2 <- lm(log(Price) ~ log(Kilometer+1) + Width + Engine + Length + `Seating Capacity` + `Fuel Tank Capacity`, data = train1)

summary(model2)

Output:

```
Call:
lm(formula = log(Price) ~ log(Kilometer + 1) + Width + Engine + 
    Length + `Seating Capacity` + `Fuel Tank Capacity`, data = train1)

Residuals:
     Min       1Q   Median       3Q      Max 
-1.93285 -0.21530  0.02364  0.28427  1.09531 

Coefficients:
                       Estimate Std. Error t value Pr(>|t|)    
(Intercept)           1.032e+01  5.369e-01  19.226  < 2e-16 ***
log(Kilometer + 1)   -2.813e-01  2.460e-02 -11.436  < 2e-16 ***
Width                 2.404e-03  2.960e-04   8.121 5.50e-15 ***
Engine                3.134e-04  7.277e-05   4.307 2.08e-05 ***
Length                4.583e-04  1.080e-04   4.245 2.71e-05 ***
`Seating Capacity`   -2.089e-01  3.104e-02  -6.731 5.72e-11 ***
`Fuel Tank Capacity`  1.537e-02  2.869e-03   5.356 1.42e-07 ***
---
Signif. codes:  0 ‘***’ 0.001 ‘**’ 0.01 ‘*’ 0.05 ‘.’ 0.1 ‘ ’ 1

Residual standard error: 0.4628 on 409 degrees of freedom
Multiple R-squared:  0.7718,    Adjusted R-squared:  0.7685 
F-statistic: 230.5 on 6 and 409 DF,  p-value: < 2.2e-16
```

From the output we can check that:

Dependent Variable: log(Price)

Independent Variable: Engine, Fuel Tank Capacity, Width, Length, log(Kilometer+1), Seating Capacity.

Min	1Q	Median	3Q	Max
-1.93	-0.22	0.02	0.28	1.10

Median near **0** → errors are centred well.

Max residual ≈ 1.1 → means the worst under/over-prediction is around **exp(1.1) ≈ 3 times** the true value (much better than earlier extreme outliers).

All predictors are **highly significant (p < 0.001)**.

No "weak" predictors left — this is a very solid model.

Multiple R-squared: 0.7718

Adjusted R-squared: 0.7685

The model now explains **~77% of the variation in log(Price)**.

F-statistic: 230.5 on 6 and 409 DF, p-value: < 2.2e-16

Model as a whole is **extremely significant**.

Linear Regression Formula:

Regression Equation:

log(Price)=10.32−0.2813*log(Kilometer+1)+0.002404*Width+0.000313*Engine+ 0.000458*Length−0.209*Seating Capacity+0.01537*Fuel Tank Capacity

So, Price Formula:

Price=exp(10.32−0.2813*log(Kilometer+1)+0.002404*Width+0.000313*Engine+ 0.000458*Length−0.209*Seating Capacity+0.01537*Fuel Tank Capacity)

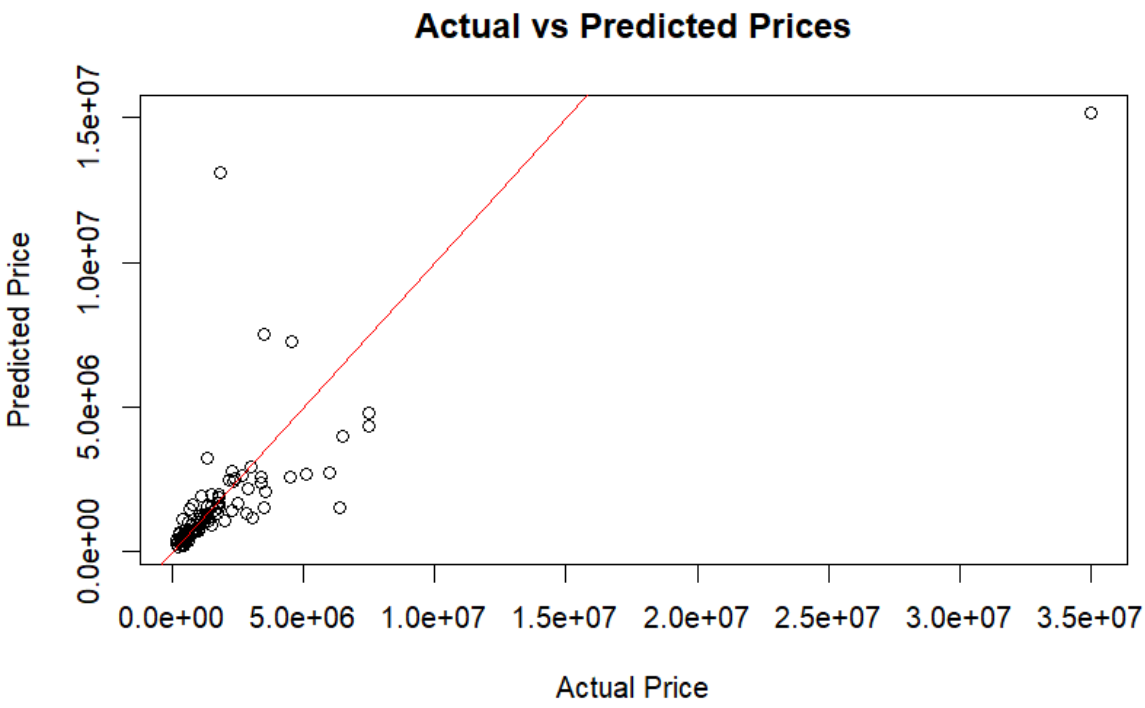
Lastly, we can run a R-squared test on the test dataset.

```
# R-squared (on test set)
ss_res <- sum((test1$Price - pred_price)^2)
ss_tot <- sum((test1$Price - mean(test1$Price))^2)
r2_test <- 1 - (ss_res/ss_tot)
```

Output: [1] 0.5363342

Plot of Actual Vs Predicted:

```
plot(test1$Price, pred_price,
    xlab = "Actual Price",
    ylab = "Predicted Price",
    main = "Actual vs Predicted Prices")
abline(0, 1, col = "red")  # ideal fit line
```


Actual vs Predicted Prices
Predicted Price
0.0e+00
5.0e+06
1.0e+07
1.5e+07
0.0e+00
5.0e+06
1.0e+07
1.5e+07
2.0e+07
2.5e+07
3.0e+07
3.5e+07
Actual Price

-----XXXXX-----